

Drawtiming Diagrams in Linux



SHAKTHI KANNAN

Drawtiming is a free and open source software command-line tool that can be used to generate digital circuit timing diagrams. It is released under the GNU General Public License (GPL). The tool accepts text file as an input from the user and generates the image output in different file formats such as PNG, JPEG, PostScript and PDF. The input needs to adhere to a specific syntax to describe different signals, their inter-relationships and transitions as described in the next section with examples.

The advantage of this tool is that you do not need to draw timing diagrams with your bare hands or mouse. Typical notations and timing diagrams found in the electrical engineering discipline are used, including arrows indicating causal relationships between signal transitions.

Software installation

Before you can try the examples, you need to install the drawtiming tool in your GNU/Linux system. Installation steps may be different from system to system. To install this software in Ubuntu 12.10, just click on Ubuntu Software Center option from desktop and search for Drawtiming to install it.

For a basic input command example to generate a clock for three periods, the syntax is written as:

```
CLOCK=0.
CLOCK=1.
CLOCK=0.
```

A period (.) is used to mark the end of each clock cycle. The first clock period lists and sets the initial states of the signals. Save the above three-line syntax as a .txt file. Here, we name the file as 'basic.txt' and save it in a folder named as, say, EFYdraw. Note that the syntax is case sensitive. To generate

the timing diagram, open EFYdraw from the terminal window using 'cd' command and enter the following command against the prompt, and you will get the output diagram as shown in Fig. 1.

```
$ drawtiming -o basic.jpg basic.txt
```

The 'tick' keyword can also indicate a clock cycle. The following syntax creates a single period clock as shown below:

```
CLOCK=tick.
```

The output diagram is shown in Fig. 2.

The order in which the signal commands are listed determines the order in which they appear in the output. Multiple signals are separated by commas as shown below:



Fig. 1: Screenshot of basic.jpg



Fig. 2: Screenshot of tick.jpg



Fig. 3: Screenshot of flag.jpg

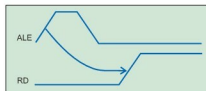


Fig. 4: Screenshot of trigger.jpg



Fig. 5: Screenshot of trigger_same.jpg

```
CLOCK=tick, FLAG=0.
```

The output diagram is shown in Fig. 3.

A change in one signal that triggers a change in another can be expressed using arrow notation (=>). In the following example (Fig. 4), when the address latch enable (ALE) signal changes state, the read signal (RD) also changes its state in the subsequent cycle. The syntax is as given below:

```
ALE=tick, RD=0.
```

```
ALE => RD=1, ALE=0.
```

If the transition is in the same clock cycle, it can be indicated in the same clock period.

For instance, the syntax for both CLK and CLK2 change in the second clock period:

```
CLK=0, CLK2=0.
```

```
CLK=tick, CLK2=1.
```

The output is as shown in Fig. 5.

If there are multiple signal changes, you can separate them with a comma. For example, LED is turned on when both the chip select (CS) and output enable (OE) signals change state; then the syntax is written as:

```
CS=0, OE=0, LED=2.
```

```
CS=1, OE=1.
```

```
CS, OE => LED=ON.
```

The output is as shown in Fig. 6

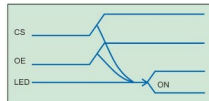


Fig. 6: Screenshot of multiple.jpg

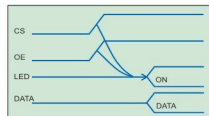


Fig. 7: Screenshot of semicolon.jpg